01. RNNs & Sequence Modeling

Overview

Recurrent networks treat sequences as ordered inputs where each step updates a hidden state. This module covers the recurrence relation, unrolling through time, backpropagation through time (BPTT), and the vanishing gradient mechanism that limits vanilla RNNs on long contexts. You will see how many-to-one, one-to-many, and encoder-decoder layouts map to real tasks such as classification, generation, and translation prep.

Lab: Lab 01.

Contents

Module track

This module starts a new track. Later modules refer back using **Module NN** labels.

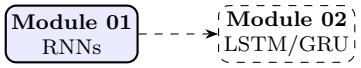

Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Motivation: Why Sequences?

Sequential data (speech, text, sensor streams, financial ticks) violates the i.i.d. assumption of feed-forward networks. A recurrent neural network (RNN) maintains a hidden state $\mathbf{h}_t$ that summarizes past context. We build on ideas developed further in **Module 02** (LSTM gate equations) and **Module 03** (attention definition).

1.1 Formal Setup

Given input sequence $\mathbf{x}_{1:T} = (x_1, \ldots, x_T)$ with $x_t \in \mathbb{R}^{d_x}$, an RNN computes

$$\mathbf{h}_t = \phi(\mathbf{W}_{xh}\,\mathbf{x}_t + \mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{b}_h)\,, \tag{1}$$

where ϕ is typically tanh or ReLU, $\mathbf{h}_0 = \mathbf{0}$, and $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$. The same weights are applied at every time step (*weight sharing* across time).

Weight sharing. Parameter count is $O(d_h^2 + d_h d_x)$ regardless of T, enabling variable-length sequences.

2 Unrolling and Backpropagation Through Time

Unrolling the recurrence over T steps yields a deep computational graph with shared parameters. Training minimizes sequence loss $\mathcal{L} = \sum_{t=1}^{T} \ell_t(\mathbf{y}_t, \hat{\mathbf{y}}_t)$ via BPTT.

2.1 Gradient Flow

The Jacobian of $\mathbf{h}_t$ w.r.t. $\mathbf{h}_{t-k}$ is a product of k local Jacobians:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-k}} = \prod_{i=t-k+1}^{t} \mathbf{W}_{hh}^{\top}\,\mathrm{diag}\big(\phi'(\mathbf{z}_i)\big)\,. \tag{2}$$

When $\|\mathbf{W}_{hh}\| < 1$ and $\phi = \tanh$, this product vanishes exponentially, the **vanishing gradient** problem. Exploding gradients are clipped in practice:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\!\left(1,\ \frac{\tau}{\|\mathbf{g}\|_2}\right). \tag{3}$$

2.2 Truncated BPTT

For long sequences we backpropagate only τ steps (e.g. $\tau = 35$), trading unbiased gradients for memory efficiency.

3 Output Heads and Many-to-Many Architectures

An output at step t is $\hat{\mathbf{y}}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$. Common patterns:

- **Many-to-one**: sentiment classification uses $\mathbf{h}_T$ only.
- **One-to-many**: image captioning feeds $\mathbf{h}_0$ from a CNN, then decodes words.
- **Seq2seq**: encoder to decoder pairs (see **Module 03** (encoder-decoder setup)).

$$P(y_{1:T} \mid x_{1:T}) = \prod_{t=1}^{T} P(y_t \mid y_{<t}, x_{1:T}). \tag{4}$$

4 Comparison with Feed-Forward Baselines

Table 1: RNN vs. feed-forward on sequence tasks.

Property	**FFN (fixed window)**	**Vanilla RNN**
Parameters vs. length T	Grows with window	Constant
Long-range dependencies	Limited window	Theoretically unbounded
Training stability	Stable	Vanishing/exploding grads
Parallelism over time	Yes (within window)	Sequential forward pass

5 Worked Example: Character-Level LM

Predict next character c_{t+1} from prefix $c_{1:t}$. Softmax output: $P(c_{t+1} = v \mid c_{1:t}) = \text{softmax}(\mathbf{W}_{out}\mathbf{h}_t)_v$. Cross-entropy loss per step:

$$\ell_t = -\log P(c_{t+1} \mid c_{1:t}). \tag{5}$$

Perplexity $\text{PPL} = \exp\left(\frac{1}{T}\sum_t \ell_t\right)$ measures compression quality.

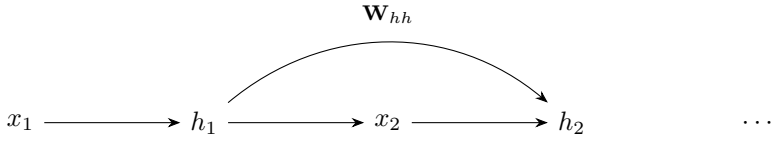

Figure 2: Unrolled RNN: shared W_{hh} links hidden states.

6 Lab Connection and Next Steps

Implement a character RNN in **Lab 01**: build the recurrence in (1), train on a text corpus, and plot loss curves. Gated architectures in **Module 02** (LSTM gate equations) address the instability noted in (2). Early sequence-to-sequence work [1] motivates attention in Module 03.

7 Truncation and Padding Strategies

Variable-length batches pad to $T_{\max}$ with mask $\mathbf{m}_t \in \{0, 1\}$:

$$\mathcal{L}_{\text{masked}} = \frac{\sum_{t=1}^{T_{\max}} m_t \, \ell_t}{\sum_{t=1}^{T_{\max}} m_t}. \tag{6}$$

Pack-padded sequences (sorted by length) minimize wasted computation on padding tokens.

7.1 Bidirectional Context Limits

Vanilla RNNs are inherently causal in many-to-one setups; encoder bidirectionality requires a second backward pass as in **Module 02** (bidirectional LSTM).

8 Extended Intuition

Vanilla RNNs work because they compress history into a fixed vector that gets updated at every step. The same weight matrices act on every time step, so the model learns transition rules rather than memorizing positions. That sharing is powerful for variable-length inputs, but it also means errors compound: a bad update at step 10 affects step 50 through repeated multiplication by $\mathbf{W}_{hh}$.

Before writing the recurrence, decide what the hidden state must remember. For sentiment, the final $\mathbf{h}_T$ often suffices (many-to-one). For next-character prediction, each $\mathbf{h}_t$ must encode the prefix $c_{1:t}$ (many-to-many). The architecture choice drives batching: many-to-one lets you take the last valid hidden state after packing; many-to-many needs aligned outputs at every step.

Weight tying between embedding and softmax output (`nn.Linear` sharing `nn.Embedding.weight`) cuts vocabulary-side parameters in half for language models. That trick does not apply to classification heads where output classes differ from input tokens. When sequences in a batch have different lengths, the hidden state at padded positions still updates unless you mask; packed sequences skip computation on pads entirely.

BPTT is not optional theory. When you call `loss.backward()` on a sequence loss, PyTorch walks the unrolled graph. Long sequences blow up memory because the graph stores every intermediate $\mathbf{h}_t$. Truncated BPTT limits how far gradients flow backward in time, which is a deliberate bias-variance trade on credit assignment. If your task needs dependencies longer than τ, stacked RNNs or gated cells (**Module 02**) are the practical next step, not infinite unrolling on a 4 GB card.

9 Numerical Walkthrough

Consider a character LM with vocabulary size $V = 64$, embedding dim $d_x = 32$, hidden size $d_h = 128$, sequence length $T = 100$, batch size $B = 32$.

At step $t = 1$: $\mathbf{x}_1 \in \mathbb{R}^{32}$ (embedded), $\mathbf{h}_0 = \mathbf{0}$.

$$\mathbf{z}_1 = \mathbf{W}_{xh}\mathbf{x}_1 + \mathbf{b}_h \in \mathbb{R}^{128}, \quad \mathbf{h}_1 = \tanh(\mathbf{z}_1). \tag{7}$$

Output logits: $\mathbf{o}_1 = \mathbf{W}_{hy}\mathbf{h}_1 + \mathbf{b}_y \in \mathbb{R}^{64}$. If the true next character is index 17, step loss is $\ell_1 = -\log \mathrm{softmax}(\mathbf{o}_1)_{17}$.

Parameter count (single layer): $32 \cdot 128 + 128 \cdot 128 + 128 + 128 \cdot 64 + 64 \approx 33$k weights. Forward activations for one batch: roughly $B \cdot T \cdot d_h = 32 \cdot 100 \cdot 128 \approx 410$k floats for hidden states alone. With Adam (2 momentum buffers), optimizer state dominates storage for small models.

Training for 5 epochs on 500k characters at batch 32, seq 100: ≈ 156 optimizer steps per epoch if non-overlapping windows. Learning rate 10^{-3} with gradient clip norm 1.0 is a sane starting point on CPU; drop to 3×10^{-4} if loss spikes.

Perplexity after epoch 1 might sit around 3.5 to 4.5 on a tiny corpus (still above unigram baseline); by epoch 5 under 2.5 indicates the RNN learned character n-gram structure. Gradient norm before clipping often peaks early around 5 to 20; sustained norms above 50 mean clip harder or reduce lr. Memory for stored BPTT graph at $T = 100$, $B = 32$, $d_h = 128$: roughly $B \times T \times d_h \times 4$ bytes for hidden activations alone ≈ 1.6 MB per layer, small until you stack layers or hit $T = 1000+$.

10 PyTorch Implementation Notes

- `nn.RNN(input_size=32, hidden_size=128, batch_first=True)` expects input shape `(B, T,` d_x`)` and returns `(output, h_n)` with `output` of shape `(B, T,` d_h`)`.
- Set `h_0 = torch.zeros(num_layers, B,` d_h`)`; wrong layer dimension is the most common shape error.
- Use `nn.utils.rnn.pack_padded_sequence` when lengths vary; do not pad with zeros and treat padded steps as real tokens without masking.
- For many-to-one classification, take `output[range(B), lengths-1, :]` or the final `h_n[-1]` after packing.
- `detach()` on $\mathbf{h}$ between truncated segments when manually splitting long sequences for TBPTT.
- Initializing $\mathbf{W}_{hh}$ with orthogonal matrices (`nn.init.orthogonal_`) sometimes delays vanishing gradients on toy copy tasks.
- `num_layers > 1` requires passing `h_0` shaped `(num_layers, B,` d_h`)`; default zeros hide shape bugs until multi-layer training.

```
emb = nn.Embedding(vocab_size, 32)
rnn = nn.RNN(32, 128, batch_first=True)
x = emb(token_ids)          # (B, T, 32)
out, h_n = rnn(x)           # out: (B, T, 128)
logits = head(out)          # (B, T, vocab_size)
loss = F.cross_entropy(
    logits.view(-1, vocab_size),
    targets.view(-1),
)
```

11 Local GPU Constraints

A GTX 1650 with 4 GB VRAM can train small character RNNs comfortably. Keep $d_h \leq 256$, $T \leq 128$, $B \leq 64$ for $V \approx 100$. If you hit OOM, reduce batch size first, then sequence length; halving B often fixes memory without hurting convergence much.

Gradient accumulation simulates larger batches: run forward-backward with $B = 16$, accumulate 4 steps, then `optimizer.step()`. RNN forward passes are sequential over T, so mixed precision (`torch.cuda.amp`) saves less than on Transformers but still helps for large embeddings. Prefer `float32` for the first debugging run; NaNs in half precision are harder to trace.

12 Debugging Checklist

- Loss flatlines: learning rate too low, or inputs not normalized; check embedding scale and gradient norms.
- Loss becomes NaN: learning rate too high or missing gradient clipping; clip global norm to 1.0 to 5.0.
- Model outputs repeated characters: insufficient training data or d_h too small for vocabulary.
- Slower than expected on GPU: batch size too small to saturate CUDA cores; increase B if memory allows.
- Validation much worse than train: overfitting; add dropout on the RNN output (`nn.Dropout(0.2)`) or reduce d_h.

- Shape mismatch after packing: forgotten `batch_first=True` or wrong `enforce_sorted=False` with unsorted lengths.

13 Practice Problems

Problem 1. Derive the number of multiply-adds for one forward pass over length T with hidden size d_h and input size d_x (ignore activations).

Hint: Each step performs two matrix-vector products: $\mathbf{W}_{xh}\mathbf{x}_t$ and $\mathbf{W}_{hh}\mathbf{h}_{t-1}$.

Problem 2. A sequence has length 500 but you truncate BPTT to $\tau = 35$. How many backward segments do you need, and what happens to gradients for dependencies longer than 35 steps?

Problem 3. Implement many-to-one sentiment: given padded token ids `(B, T)` and lengths `(B,)`, return logits `(B, num_classes)` using only the last valid hidden state.

Problem 4. Compare perplexity of a uniform baseline (PPL $= V$) to your trained LM on a held-out chunk. What PPL indicates the model beats random guessing?

13.1 Sequence Batch Layout

When building batches from variable-length text, sort by length descending before `pack_padded_sequence` so the first batch item has the longest sequence (CuDNN efficiency hint). Document your `PAD_IDX` in the tokenizer module; accidental training on pad tokens inflates loss without improving generation. For evaluation, compute perplexity only on non-pad positions by masking loss with `ignore_index=PAD_IDX`.

See **Lab 01** for the full training loop; gated cells in **Module 02** address long-range gradient decay from **Module 01**.

Source certification

This module lists where each core definition, equation, figure, and summary table comes from. Pedagogical simplifications (lab batch sizes, epoch counts, illustrative plots) are labeled in extension sections and are not claimed as optimal production settings.

Table 3: Certified topic map for Module 01.

Topic	Primary source	Where in this PDF
RNN recurrence and hidden-state update	[3]	Eq. 1
BPTT gradient product / vanishing gradients	[6]	Eq. 2
Seq2seq encoder-decoder motivation	[7]	Section 3
LSTM motivation (forward reference)	[4]	Section 6

Full bibliographic entries appear below. Figures are schematic unless a caption cites a specific paper figure. If a statement is not listed here, treat it as general ML practice or lab-specific guidance, not a cited theorem.

References

[1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.

[2] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *EMNLP*, 2014.

[3] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.

[4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[5] Rafal Jozefowicz, Wojciech Zaremba, and Ilya Sutskever. An empirical exploration of recurrent network architectures. In *ICML*, 2015.

[6] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *ICML*, 2013.

[7] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *NeurIPS*, 2014.